

Experiment 3: Creation of a Simple Network Topology

Using Open-Source Network Virtualization Tools

Aim / Objective

To create and configure a simple network topology using an open-source network virtualization tool and verify communication between the virtual network nodes.

Requirements

- Ubuntu Linux Virtual Machine
- VirtualBox
- Mininet
- Create simple Network Topology
- ping and basic Linux networking commands
- After completion, stop the network.

Practical Procedure and Evidence

Photo 1 – Checking Mininet Version

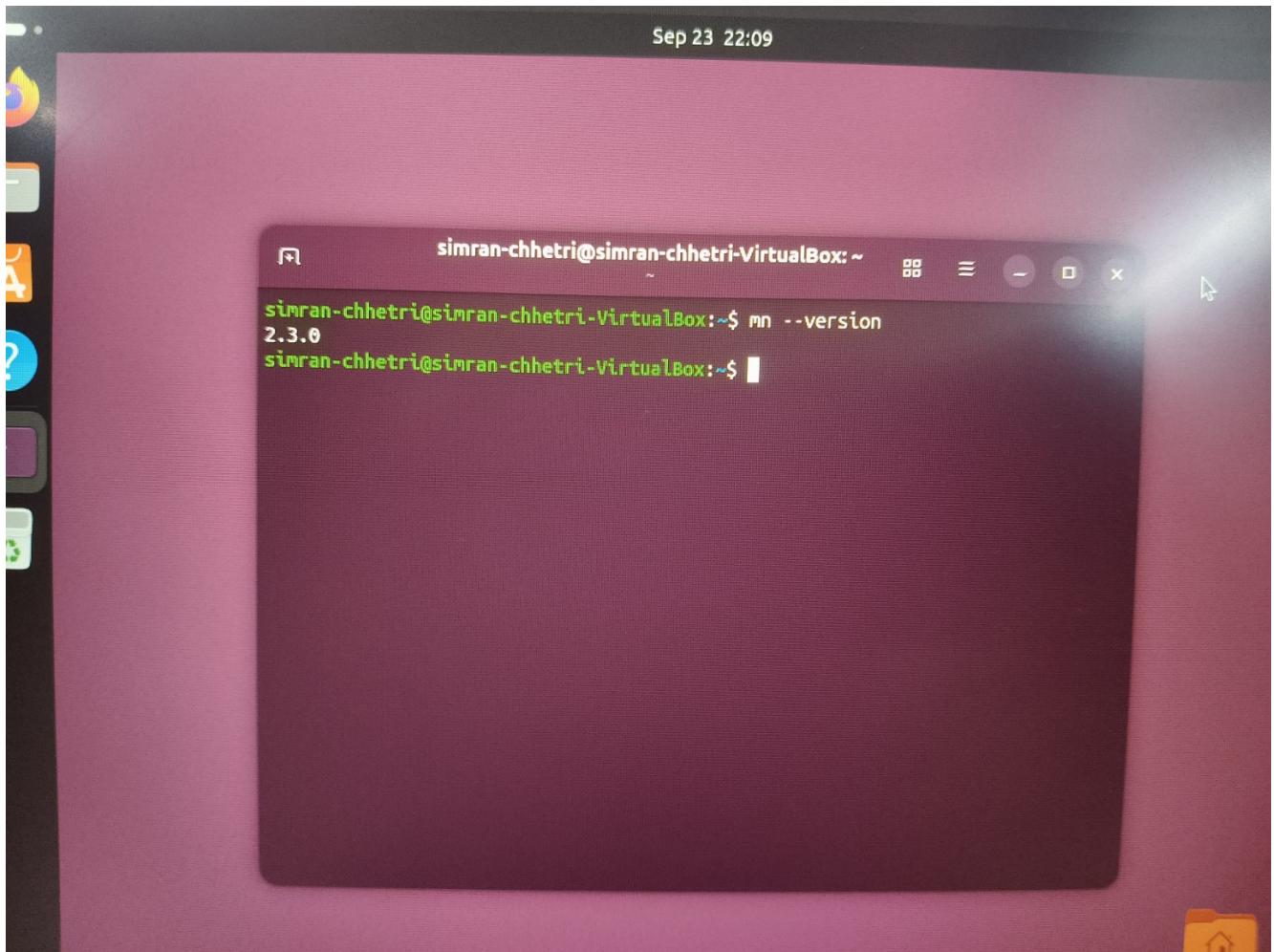
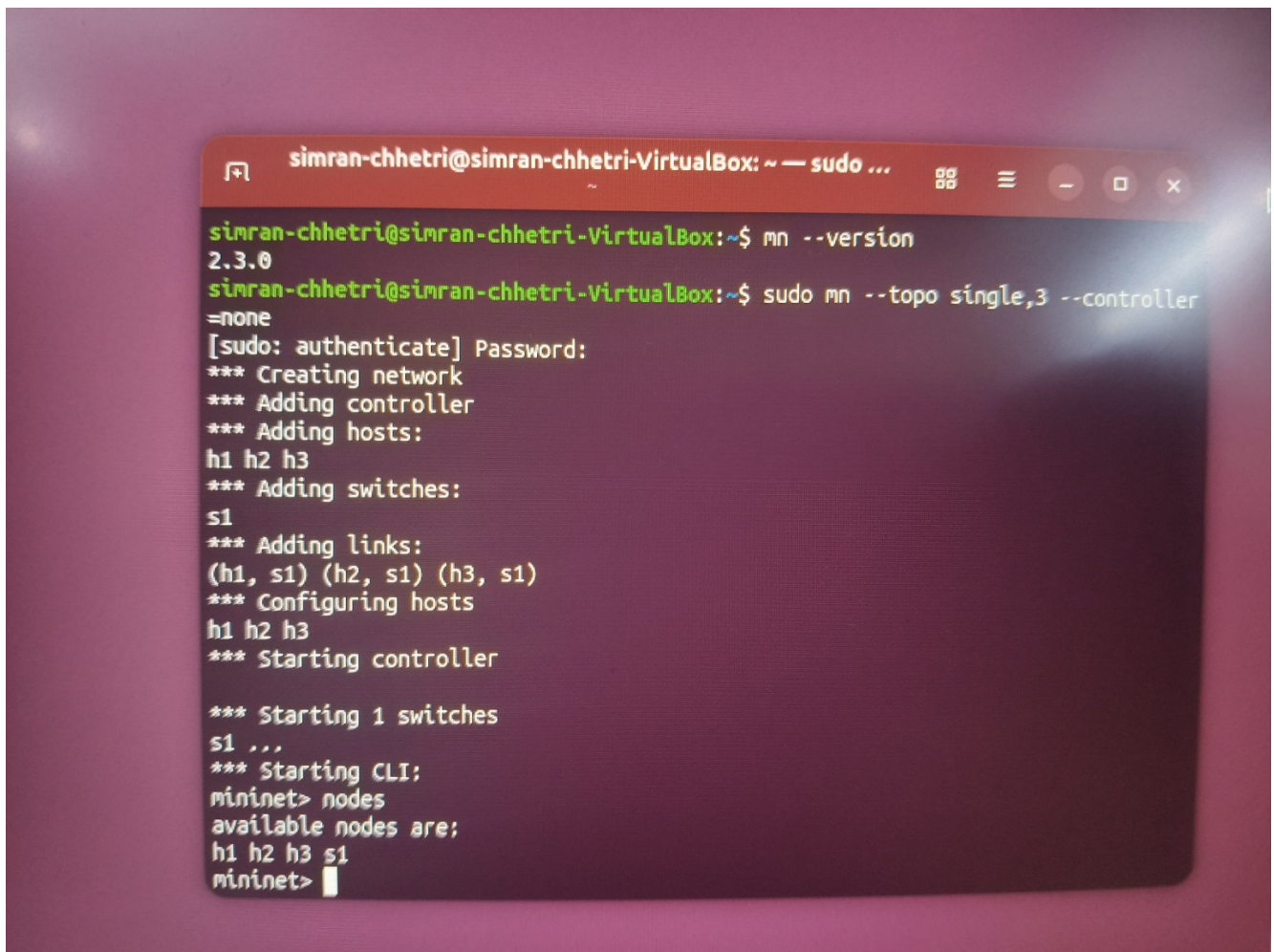


Figure 1: Practical screenshot

Command: mn --version

Observation: Mininet version **2.3.0** is installed successfully.

Photo 2 – Creating the Simple Network Topology



```
simran-chhetri@simran-chhetri-VirtualBox: ~ — sudo ...  
simran-chhetri@simran-chhetri-VirtualBox:~$ mn --version  
2.3.0  
simran-chhetri@simran-chhetri-VirtualBox:~$ sudo mn --topo single,3 --controller  
=none  
[sudo: authenticate] Password:  
*** Creating network  
*** Adding controller  
*** Adding hosts:  
h1 h2 h3  
*** Adding switches:  
s1  
*** Adding links:  
(h1, s1) (h2, s1) (h3, s1)  
*** Configuring hosts  
h1 h2 h3  
*** Starting controller  
  
*** Starting 1 switches  
s1 ...  
*** Starting CLI:  
mininet> nodes  
available nodes are:  
h1 h2 h3 s1  
mininet>
```

Figure 2: Practical screenshot

Command: `sudo mn --topo single,3 --controller=none`

Observation: A single-switch topology with three hosts is created. The nodes command verifies **h1, h2, h3, s1**.

Photo 3 – Testing Connectivity with pingall

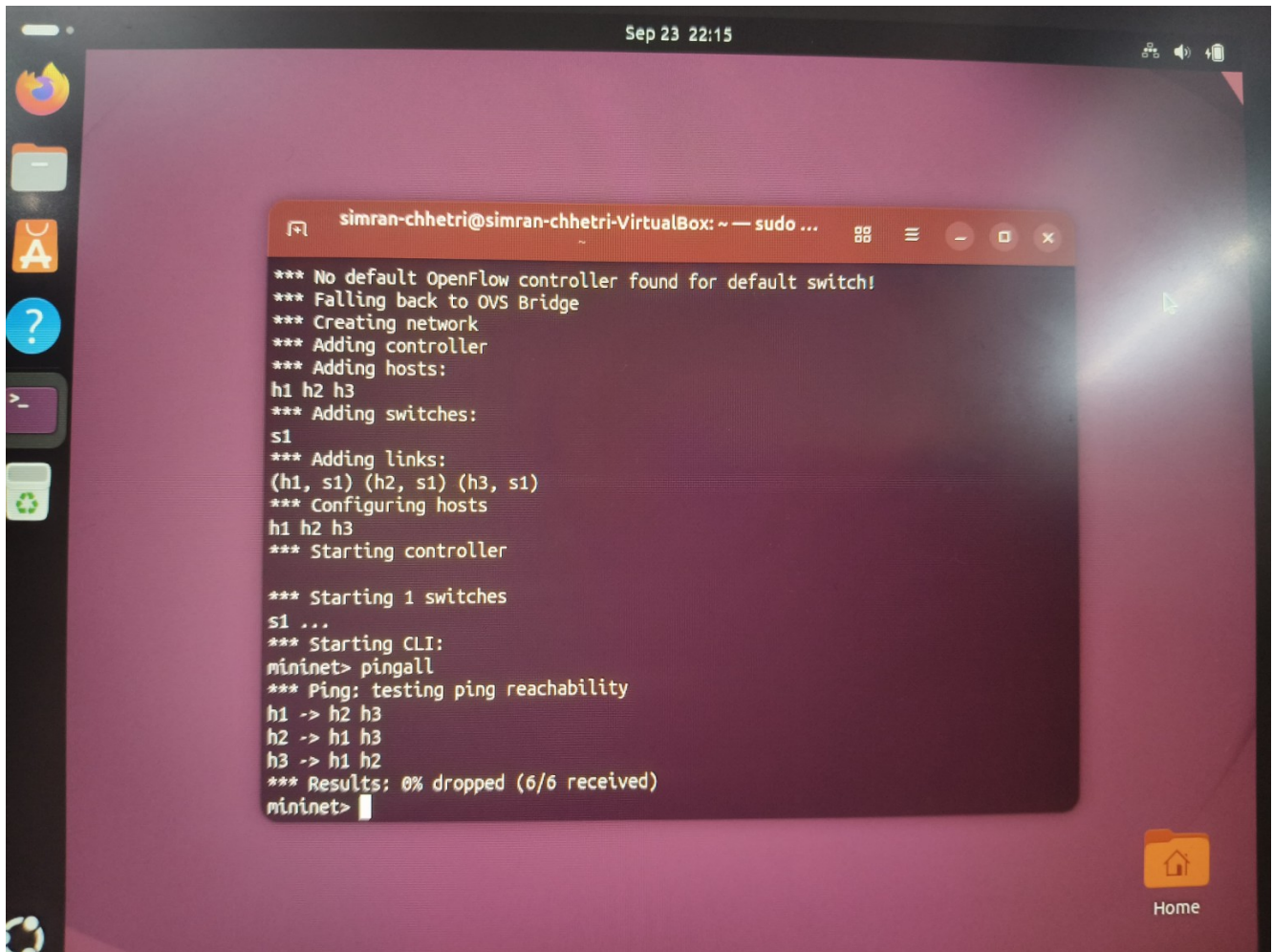
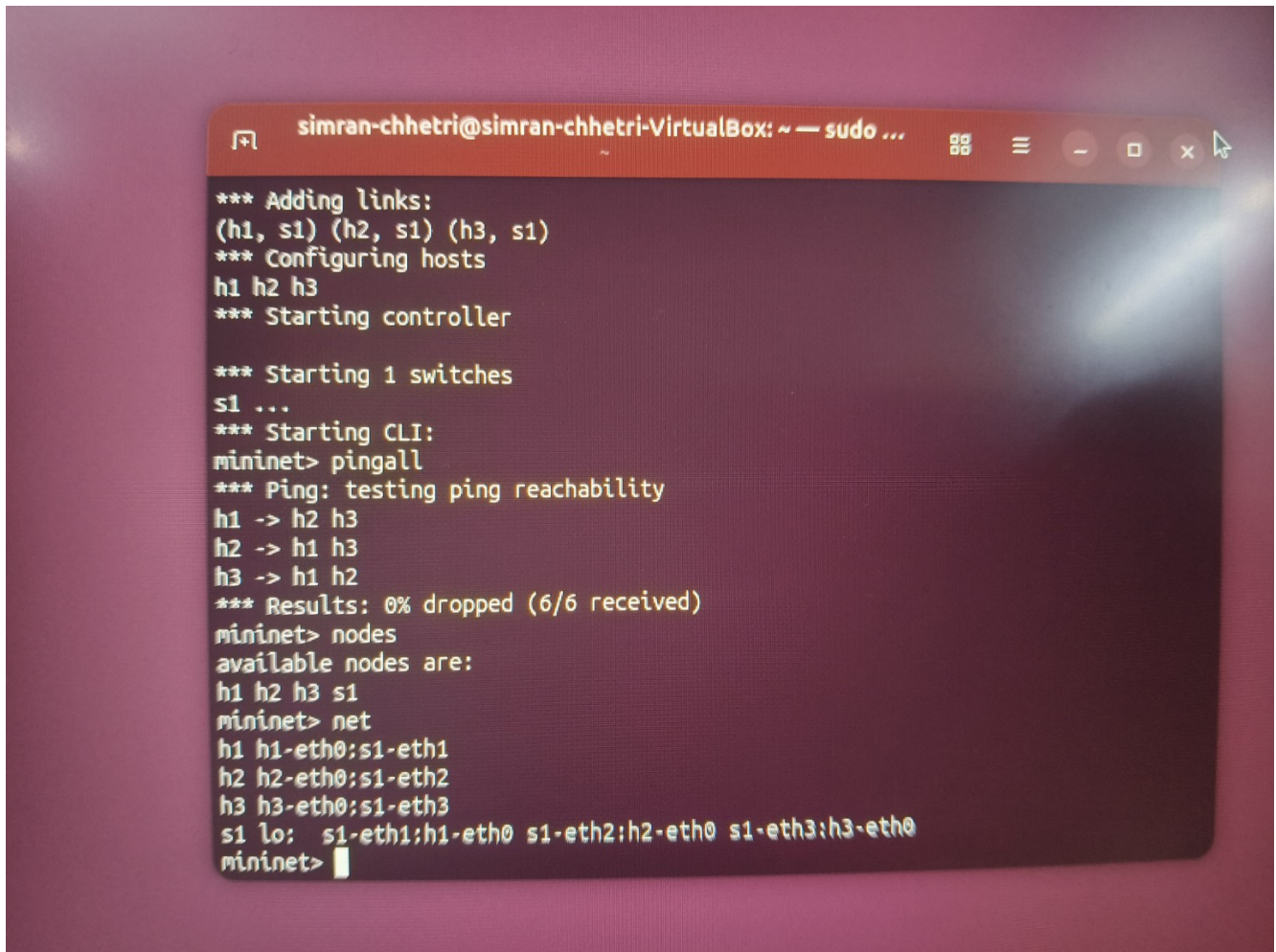


Figure 3: Practical screenshot

Command: pingall

Observation: All hosts are reachable. The result shows **0% dropped (6/6 received)**, confirming successful connectivity.

Photo 4 – Verifying the Topology

A screenshot of a terminal window titled 'simran-chhetri@simran-chhetri-VirtualBox: ~ — sudo ...'. The terminal displays the output of several commands used to set up and verify a network topology. The output shows the addition of links, configuration of hosts, starting of a controller and switch, and a successful pingall test. The 'net' command shows the network connections between hosts and the switch.

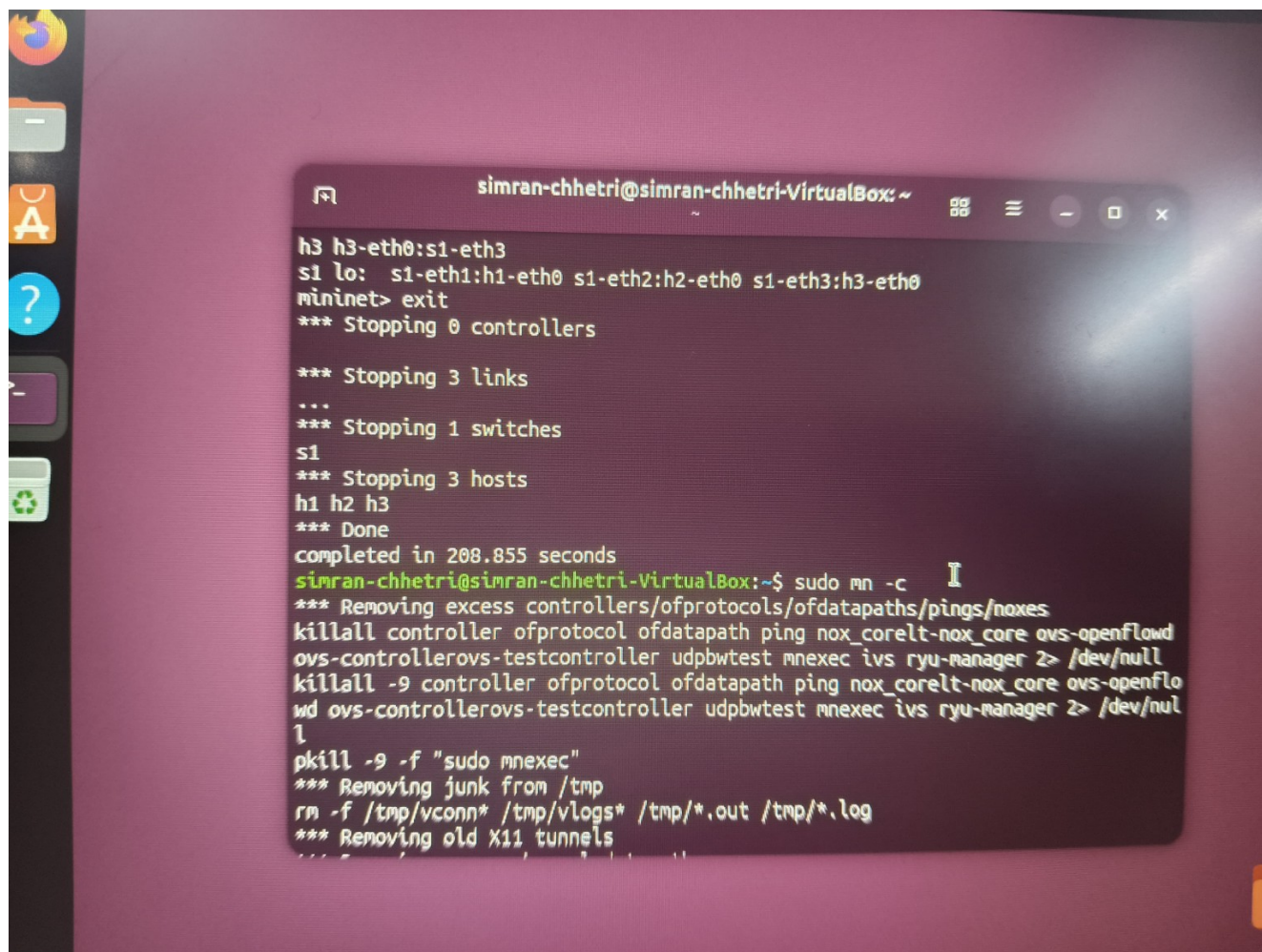
```
simran-chhetri@simran-chhetri-VirtualBox: ~ — sudo ...  
*** Adding links:  
(h1, s1) (h2, s1) (h3, s1)  
*** Configuring hosts  
h1 h2 h3  
*** Starting controller  
  
*** Starting 1 switches  
s1 ...  
*** Starting CLI:  
mininet> pingall  
*** Ping: testing ping reachability  
h1 -> h2 h3  
h2 -> h1 h3  
h3 -> h1 h2  
*** Results: 0% dropped (6/6 received)  
mininet> nodes  
available nodes are:  
h1 h2 h3 s1  
mininet> net  
h1 h1-eth0:s1-eth1  
h2 h2-eth0:s1-eth2  
h3 h3-eth0:s1-eth3  
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0  
mininet> |
```

Figure 4: Practical screenshot

Commands: pingall, nodes, net

Observation: Nodes **h1**, **h2**, **h3**, **s1** are present and links **h1-eth0:s1-eth1**, **h2-eth0:s1-eth2**, **h3-eth0:s1-eth3** are verified. Pingall reports **0% dropped (6/6 received)**.

Photo 5 – Exiting Mininet and Starting Cleanup

A photograph of a computer screen showing a terminal window. The terminal window has a title bar that reads "simran-chhetri@simran-chhetri-VirtualBox: ~". The terminal output shows the user typing "h3 h3-eth0:s1-eth3", "s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0", and "mininet> exit". This is followed by a series of status messages: "*** Stopping 0 controllers", "*** Stopping 3 links", "...", "*** Stopping 1 switches", "s1", "*** Stopping 3 hosts", "h1 h2 h3", and "*** Done". A completion message states "completed in 208.855 seconds". The user then types "simran-chhetri@simran-chhetri-VirtualBox:~\$ sudo mn -c". This triggers a long list of processes to be killed, including "controller", "ofprotocol", "ofdatapath", "ping", "nox_corelt-nox_core", "ovs-openflowd", "ovs-controllerovs-testcontroller", "udpbwtest", "mnexec", and "ivs ryu-manager". The terminal shows "killall" and "pkill" commands being executed. Finally, cleanup commands are shown: "*** Removing junk from /tmp", "rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log", and "*** Removing old X11 tunnels".

```
simran-chhetri@simran-chhetri-VirtualBox: ~
h3 h3-eth0:s1-eth3
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
mininet> exit
*** Stopping 0 controllers

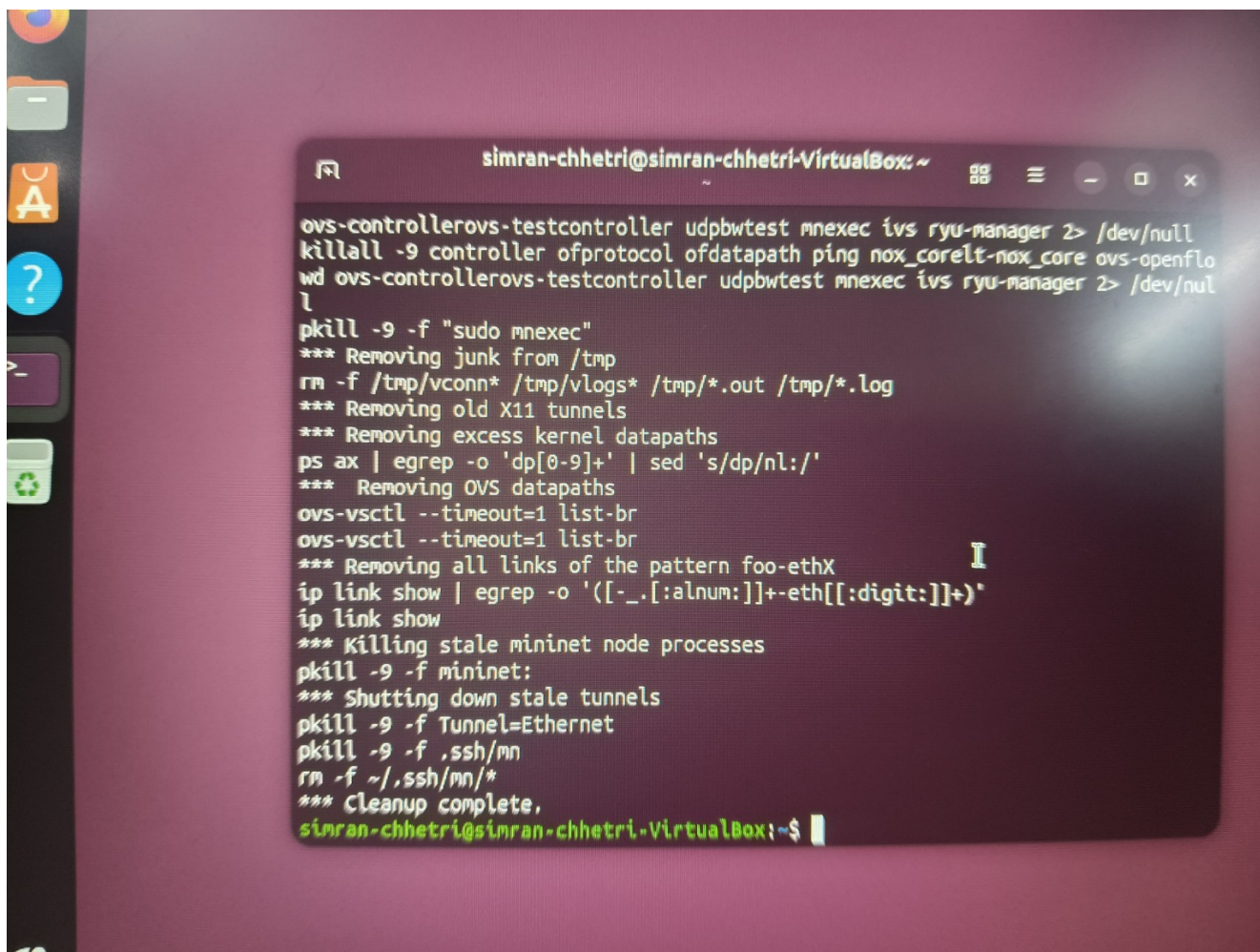
*** Stopping 3 links
...
*** Stopping 1 switches
s1
*** Stopping 3 hosts
h1 h2 h3
*** Done
completed in 208.855 seconds
simran-chhetri@simran-chhetri-VirtualBox:~$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd
ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflo
wd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/nul
l
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
```

Figure 5: Practical screenshot

Command: exit followed by `sudo mn -c`

Observation: The Mininet CLI is exited and the cleanup process is started to remove remaining Mininet resources.

Photo 6 – Cleanup Completed Successfully



```
simran-chhetri@simran-chhetri-VirtualBox: ~  
ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null  
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflo  
wd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/nul  
l  
pkill -9 -f "sudo mnexec"  
*** Removing junk from /tmp  
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log  
*** Removing old X11 tunnels  
*** Removing excess kernel datapaths  
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'  
*** Removing OVS datapaths  
ovs-vsctl --timeout=1 list-br  
ovs-vsctl --timeout=1 list-br  
*** Removing all links of the pattern foo-ethX  
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'  
ip link show  
*** Killing stale mininet node processes  
pkill -9 -f mininet:  
*** Shutting down stale tunnels  
pkill -9 -f Tunnel=Ethernet  
pkill -9 -f .ssh/mn  
rm -f ~/.ssh/mn/*  
*** Cleanup complete.  
simran-chhetri@simran-chhetri-VirtualBox: ~$
```

Figure 6: Practical screenshot

Command: sudo mn -c

Observation: Mininet resources are removed and **cleanup complete** is displayed, confirming successful cleanup.

Result / Conclusion

The simple Mininet network topology was successfully created with three hosts (h1, h2, h3) connected to switch s1. Connectivity was verified using `pingall`, showing 0% packet loss (6/6 received). The topology and network links were verified, and the Mininet environment was stopped and cleaned successfully using `sudo mn -c`.